Buổi 5

Constructor và Destructor

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Constructor – khởi động đối tượng

1.1 Khái niệm constructor
1.2 Constructor mặc định và constructor có tham số
1.3 Đa năng hóa constructor và danh sách khởi tạo

2. Destructor – hủy đối tượng

2.1 Khái niệm destructor
2.2 Ví dụ: lớp mảng động tự giải phóng
2.3 Khi nào constructor và destructor được gọi?
2.4 Phân tích thứ tự gọi constructor và destructor
2.5 Constructor, destructor và cấp phát động

3. Sử dụng thành viên dữ liệu và hàm thành viên

3.1 Sử dụng thành viên dữ liệu và hàm thành viên một cách an toàn
3.2 Trả về tham chiếu tới thành viên dữ liệu private – một cái bẫy
3.3 Phép gán bằng sao chép thành viên mặc định

4. Tổng kết và thực hành

4.1 Tóm tắt buổi 5
4.2 Bài thực hành buổi 5 – Theo dõi thứ tự gọi constructor/destructor
4.3 Câu hỏi và bài tập
4.4 Bài tập dự đoán kết quả

1.1 Khái niệm constructor

Constructor (hàm thiết lập, hàm khởi tạo)

Hàm thành viên đặc biệt **cùng tên với lớp**, **không có kiểu trả về** (kể cả void), được **tự động gọi** mỗi khi một đối tượng của lớp được tạo ra, nhằm **khởi tạo** các thành viên dữ liệu.

- **Đặc điểm:**
  - Tên trùng tên lớp; không có kiểu trả về; thường đặt ở vùng **public**.
  - **Tự động** được gọi – người dùng không gọi tường minh như hàm thường.
  - Có thể có tham số, tham số mặc định, và có thể **đa năng hóa** (nhiều constructor).
  - Nếu lớp **không** khai báo constructor nào, trình biên dịch tự sinh **constructor mặc định** rỗng (không khởi tạo gì ⇒ dữ liệu rác).
- **Vì sao cần?** Bảo đảm đối tượng **hợp lệ ngay từ lúc sinh ra**; không còn tình trạng "quên gọi Set" như buổi 4.

```
class Time
{  public:
      Time();                        // constructor
      ...
   private:
      int Hour, Minute, Second;
};
Time::Time()                          // không có kiểu trả về
{  Hour = Minute = Second = 0;  }
int main()
{  Time t;          // Time() tự động chạy ⇒ 00:00:00
   Time a[10];      // gọi Time() 10 lần
   Time *p = new Time;  // new gọi Time()
   delete p;
}
```

Phân biệt

`Time t;` – tạo đối tượng, gọi constructor mặc định. `Time t();` – **không** tạo đối tượng mà khai báo một **hàm** tên t trả về Time (lỗi kinh điển "most vexing parse").

1.2 Constructor mặc định và constructor có tham số

- **Constructor mặc định:** constructor gọi được **không cần đối số** – không có tham số, hoặc mọi tham số đều có giá trị mặc định.
- **Constructor có tham số:** cho phép khởi tạo đối tượng với giá trị cụ thể ngay khi khai báo.

```
class Time
{  public:
      Time(int h = 0, int m = 0, int s = 0)
      {  SetTime(h, m, s);  }       // tận dụng kiểm tra
      void SetTime(int h, int m, int s);
      void PrintMilitary();
   private:
      int Hour, Minute, Second;
};
```

- Một constructor với **tham số mặc định** thay cho 4 constructor: Time(), Time(h), Time(h, m), Time(h, m, s).
- Gọi lại SetTime trong constructor ⇒ **kiểm tra hợp lệ ở một chỗ duy nhất**.

Ví dụ 5.1 – Các cách khởi tạo (theo giáo trình)

```
Time t1,                  // tất cả mặc định
     t2(2),               // giờ
     t3(21, 34),          // giờ, phút
     t4(12, 25, 42),      // giờ, phút, giây
     t5(27, 74, 99);      // giá trị sai
t1.PrintMilitary(); ...
```

```
00:00 02:00 21:34 12:25 00:00
```

- t5(27, 74, 99): SetTime đặt các giá trị sai về 0 ⇒ **00:00**.

Ví dụ 5.2 – Các dạng khai báo khác

```
Time *p = new Time(8, 15);   // 08:15
Time arr[3];                 // cần constructor mặc định
Time t6 = Time(9, 30);       // tạo tạm rồi khởi tạo t6
```

- Nếu lớp **chỉ** có `Time(int h, int m, int s)` (không mặc định) ⇒ `Time arr[3];` và `Time t;` **lỗi**: no matching function for call to 'Time::Time()'.

1.3 Đa năng hóa constructor và danh sách khởi tạo

- Một lớp có thể có **nhiều constructor** khác tham số (đa năng hóa – buổi 3):

```
class PhanSo
{  public:
      PhanSo()             { Tu = 0; Mau = 1; }
      PhanSo(int t)        { Tu = t; Mau = 1; }
      PhanSo(int t, int m) { Tu = t; Mau = (m != 0) ? m : 1;
RutGon(); }
      ...
};
PhanSo a;          // 0/1
PhanSo b(5);       // 5/1
PhanSo c(6, -8);   // -3/4
```

- Lưu ý tránh nhập nhằng: nếu có cả `PhanSo()` và `PhanSo(int t = 0)` thì `PhanSo a;` sẽ báo **ambiguous**.

- **Danh sách khởi tạo thành viên** (member initializer list) – khởi tạo thành viên **trước** khi vào thân constructor:

```
PhanSo::PhanSo(int t, int m) : Tu(t), Mau(m)
{  if (Mau == 0) Mau = 1;
   RutGon();
}
```

- **Bắt buộc** dùng danh sách khởi tạo cho:
  - Thành viên **const** và thành viên **tham chiếu** (không gán được trong thân hàm).
  - Thành viên là **đối tượng của lớp khác** không có constructor mặc định (buổi 6).
  - Lớp cơ sở khi kế thừa (buổi 9).
- Thứ tự khởi tạo theo **thứ tự khai báo** thành viên trong lớp, **không** theo thứ tự viết trong danh sách.

Destructor (hàm hủy)

Hàm thành viên đặc biệt có tên là **dấu ngã + tên lớp** (`~Time`), **không** có tham số, **không** có kiểu trả về, được **tự động gọi** khi đối tượng bị hủy, để thực hiện "dọn dẹp" (giải phóng bộ nhớ động, đóng tập tin…) trước khi bộ nhớ của đối tượng được thu hồi.

- **Đặc điểm:**
  - Mỗi lớp có **đúng một** destructor (không đa năng hóa được vì không có tham số).
  - Nếu không khai báo, trình biên dịch tự sinh destructor rỗng.
  - Lớp **không** cấp phát tài nguyên (chỉ có int, double…) thì thường **không cần** viết destructor.
  - Lớp có **con trỏ tới vùng nhớ cấp phát động ⇒ bắt buộc** viết destructor để `delete`, nếu không sẽ rò rỉ bộ nhớ.
- **Thứ tự bên trong khi hủy:** thân destructor chạy **trước**, sau đó các thành viên dữ liệu (và lớp cơ sở – buổi 9) mới bị hủy.

Ví dụ 5.2a – Những việc thường làm trong destructor

- `delete` / `delete[]` các vùng nhớ đã cấp phát trong constructor.
- Đóng tập tin đã mở (`fclose`), đóng kết nối mạng/CSDL.
- Ghi nhật ký, cập nhật bộ đếm số đối tượng đang tồn tại (buổi 6 – thành viên static).

2.2 Ví dụ: lớp mảng động tự giải phóng

```
class Mang
{  public:
      Mang(int n)
      {  N = n;  A = new int[N];
         for (int i = 0; i < N; i++) A[i] = 0;
         cout << "  [cap phat " << N << " int]\n";
      }
      ~Mang()
      {  delete[] A;
         cout << "  [giai phong " << N << " int]\n";
      }
      void Set(int i, int x) { if (i >= 0 && i < N) A[i] = x;
}
      int  Get(int i) { return A[i]; }
   private:
      int *A;  int N;
};
```

```
int main()
{  {  Mang m(5);  m.Set(2, 9);
      cout << "  m[2] = " << m.Get(2) << endl;
   }                          // m ra khỏi phạm vi ⇒ ~Mang()
   cout << "Sau khoi lenh" << endl;
}
```

```
  [cap phat 5 int]
  m[2] = 9
  [giai phong 5 int]
Sau khoi lenh
```

- Biến m là đối tượng **cục bộ** trong khối `{ }` ⇒ khi ra khỏi khối, destructor **tự động** chạy ⇒ không thể quên giải phóng.
- So với C: phải nhớ gọi `free(a)` ở **mọi** nhánh thoát của hàm (return sớm, lỗi…).

Cẩn thận

Destructor **không** được gọi tường minh như `m.~Mang();` trong chương trình thông thường – nó sẽ bị gọi **lần nữa** khi m ra khỏi phạm vi ⇒ giải phóng hai lần.

2.3 Khi nào constructor và destructor được gọi?

Loại đối tượng	Constructor được gọi	Destructor được gọi
Toàn cục	Trước khi main bắt đầu	Khi chương trình kết thúc (sau main)
Cục bộ tự động	Khi chương trình chạy đến lệnh khai báo	Khi ra khỏi khối `{ }` chứa nó
Cục bộ static	Lần **đầu tiên** chạy đến lệnh khai báo (chỉ một lần)	Khi chương trình kết thúc
Cấp phát động	Khi thực hiện `new`	Khi thực hiện `delete` (nếu không delete ⇒ **không bao giờ**)
Tham số truyền giá trị	Khi gọi hàm (copy constructor – buổi 8)	Khi hàm kết thúc
Đối tượng tạm	Khi biểu thức tạo ra nó	Cuối câu lệnh chứa biểu thức

- **Quy tắc chung:** các đối tượng cùng loại bị hủy theo **thứ tự ngược** với thứ tự tạo (giống ngăn xếp – LIFO).

Ví dụ 5.4 – Lớp theo dõi (giáo trình): in thông báo trong constructor và destructor

```
class CreateAndDestroy
{  public:
      CreateAndDestroy(int value)
      {  data = value;  cout << "Object " << data << "   constructor";  }
      ~CreateAndDestroy()
      {  cout << "Object " << data << "   destructor " << endl;  }
   private:
      int data;
};
```

2.4 Phân tích thứ tự gọi constructor và destructor

```
CreateAndDestroy first(1);          // toàn cục
int main()
{  cout << "   (global created before main)\n";
   CreateAndDestroy second(2);      // cục bộ
   cout << "   (local automatic in main)\n";
   static CreateAndDestroy third(3); // static
   cout << "   (local static in main)\n";
   Create();
   CreateAndDestroy fourth(4);
   cout << "   (local automatic in main)\n";
   return 0;
}
void Create()
{  CreateAndDestroy fifth(5);
   cout << "   (local automatic in Create)\n";
   static CreateAndDestroy sixth(6);
   cout << "   (local static in Create)\n";
   CreateAndDestroy seventh(7);
   cout << "   (local automatic in Create)\n";
}
```

```
Object 1   constructor   (global created before main)
Object 2   constructor   (local automatic in main)
Object 3   constructor   (local static in main)
Object 5   constructor   (local automatic in Create)
Object 6   constructor   (local static in Create)
Object 7   constructor   (local automatic in Create)
Object 7   destructor
Object 5   destructor
Object 4   constructor   (local automatic in main)
Object 4   destructor
Object 2   destructor
Object 6   destructor
Object 3   destructor
Object 1   destructor
```

- **7, 5** hủy khi Create **kết thúc** (ngược thứ tự tạo); **6** là static nên **còn sống**.
- Khi main kết thúc: hủy cục bộ **4, 2**; rồi các static **6, 3** (ngược thứ tự tạo); cuối cùng toàn cục **1**.
- Nếu gọi Create() **lần hai**: tạo lại 5, 7 nhưng **không** tạo lại 6.

- `new` = cấp phát bộ nhớ **+ gọi constructor**; `delete` = **gọi destructor** + giải phóng bộ nhớ (khác malloc/free – buổi 2).

```
Time *p = new Time(8, 15);   // Time(8, 15, 0) được gọi
p->PrintMilitary();          // 08:15
delete p;                    // ~Time() được gọi
Time *ds = new Time[3];      // Time() gọi 3 lần
delete[] ds;                 // ~Time() gọi 3 lần
```

- `delete ds;` (thiếu `[]`) với mảng đối tượng ⇒ chỉ **một** destructor được gọi, các phần tử còn lại không được dọn dẹp ⇒ lỗi.

Ví dụ 5.5 – Rò rỉ khi quên delete con trỏ tới đối tượng

```
void XuLy()
{  Mang *p = new Mang(1000);   // cấp phát 1000 int
   p->Set(0, 1);
}                              // p mất, ~Mang KHÔNG được
gọi
for (int i = 0; i < 1000000; i++)
   XuLy();                     // rò rỉ ≈ 4 GB!
```

- Sửa: `delete p;` cuối hàm, hoặc dùng đối tượng **cục bộ** `Mang m(1000);` ⇒ destructor tự gọi khi ra khỏi hàm.

- **Nguyên lý RAII** (Resource Acquisition Is Initialization): **constructor lấy tài nguyên, destructor trả tài nguyên** ⇒ dùng đối tượng cục bộ thì tài nguyên (bộ nhớ, tập tin, khóa) luôn được giải phóng đúng lúc. Lớp `string`, `vector`, `fstream` đều theo nguyên lý này.

- **Hàm set** kiểm tra dữ liệu và có thể **báo lỗi** cho nơi gọi bằng giá trị trả về:

```
class Time
{  public:
      Time(int = 0, int = 0, int = 0);
      int SetHour(int h);       // trả 0 nếu sai
      int SetMinute(int m);
      int SetSecond(int s);
      int GetHour()   { return Hour; }
      int GetMinute() { return Minute; }
      int GetSecond() { return Second; }
      ...
};
int Time::SetHour(int h)
{  if (h < 0 || h >= 24) return 0;
   Hour = h;
   return 1;
}
```

Ví dụ 5.6 – Hàm tăng phút dùng các hàm truy cập

```
void TangPhut(Time &t, int Count)   // hàm ngoài lớp
{  for (int i = 1; i <= Count; i++)
   {  t.SetMinute((t.GetMinute() + 1) % 60);
      if (t.GetMinute() == 0)
         t.SetHour((t.GetHour() + 1) % 24);
   }
}
Time t(23, 58);
TangPhut(t, 3);       // 23:58 → 23:59 → 00:00 → 00:01
t.PrintMilitary();    // 00:01
```

- Hàm **ngoài lớp** không truy cập trực tiếp Hour, Minute nhưng vẫn làm được việc nhờ get/set – và **không thể** làm hỏng đối tượng.

- **Lời khuyên:** ngay cả hàm thành viên cũng nên dùng set/get thay vì truy cập trực tiếp nếu muốn tập trung việc kiểm tra ở một chỗ.

3.2 Trả về tham chiếu tới thành viên dữ liệu private – một cái bẫy

```
class Time
{  public:
      ...
      int &BadSetHour(int hh)        // trả về THAM CHIẾU
      {  Hour = (hh >= 0 && hh < 24) ? hh : 0;
         return Hour;
      }
   private:
      int Hour, Minute, Second;
};
int main()
{  Time t;
   int &HourRef = t.BadSetHour(20);
   cout << "Hour before modification: " << HourRef;
   HourRef = 30;                    // sửa trực tiếp Hour!
   cout << "\nHour after modification: "
        << t.GetHour();
   t.BadSetHour(12) = 74;           // gán vào lời gọi hàm
   cout << "\nHour after BadSetHour(12) = 74: " <<
t.GetHour();
}
```

```
Hour before modification: 20
Hour after modification: 30
Hour after BadSetHour(12) = 74: 74
```

- Hàm trả về `int &` ⇒ người gọi nhận được **bí danh** của thành viên private Hour ⇒ gán **30**, **74** (giá trị không hợp lệ) mà **không qua kiểm tra** nào.
- **Phá vỡ đóng gói** – trình biên dịch không báo lỗi.

Quy tắc

Không trả về tham chiếu (hoặc con trỏ) **không hằng** tới thành viên dữ liệu private. Nếu muốn tránh sao chép dữ liệu lớn khi đọc, trả về **tham chiếu hằng** `const int &GetHour() const;` (buổi 6).

3.3 Phép gán bằng sao chép thành viên mặc định

- Toán tử gán `=` có thể dùng để gán một đối tượng cho đối tượng **cùng lớp**; mặc định thực hiện **sao chép từng thành viên** (memberwise copy).

Ví dụ 5.8 – Lớp Date (giáo trình)

```
class Date
{  public:
      Date(int d = 1, int m = 1, int y = 1990)
      {  Day = d; Month = m; Year = y;  }
      void Print()
      {  cout << Day << "/" << Month << "/" << Year;  }
   private:
      int Day, Month, Year;
};
int main()
{  Date d1(7, 4, 1993), d2;
   cout << "d1 = "; d1.Print();
   cout << "\nd2 = "; d2.Print();
   d2 = d1;             // sao chép từng thành viên
   cout << "\nSau d2 = d1, d2 = "; d2.Print();
}
```

```
d1 = 7/4/1993
d2 = 1/1/1990
Sau d2 = d1, d2 = 7/4/1993
```

- Đối tượng còn được **truyền giá trị** cho hàm và **trả về** từ hàm theo cùng cơ chế sao chép.

Ví dụ 5.9 – Nguy hiểm với lớp có con trỏ (lớp Mang)

```
Mang a(3), b(3);
a.Set(0, 5);
b = a;          // b.A = a.A (sao chép CON TRỎ)
b.Set(0, 99);
cout << "a[0] = " << a.Get(0) << endl;
```

```
a[0] = 99
ERROR: AddressSanitizer: attempting double-free
```

- `b = a` chỉ sao chép **địa chỉ** ⇒ a.A và b.A cùng trỏ **một** vùng nhớ ("sao chép nông"):
  - Sửa b làm **thay đổi** a (a[0] = 99).
  - Vùng nhớ cũ của b bị **rò rỉ**.
  - Khi kết thúc, ~Mang của a và b cùng `delete[]` **một** vùng ⇒ **giải phóng hai lần** ⇒ crash.
- Giải pháp: **sao chép sâu** – tự viết copy constructor và operator= (buổi 8).

4.1 Tóm tắt buổi 5

Khái niệm	Ghi nhớ
Constructor	Cùng tên lớp, không kiểu trả về, tự gọi khi tạo đối tượng; đa năng hóa được; có tham số mặc định
Constructor mặc định	Gọi được không cần đối số; cần cho mảng đối tượng; tự sinh nếu lớp không có constructor nào
Danh sách khởi tạo	`: Tu(t), Mau(m)` – bắt buộc cho const, tham chiếu, đối tượng thành viên, lớp cơ sở
Destructor	`~TenLop()`, không tham số, một lớp một destructor; giải phóng tài nguyên
Thứ tự gọi	Toàn cục trước main; cục bộ theo khối; static một lần; hủy **ngược** thứ tự tạo
new / delete	new gọi constructor; delete gọi destructor; mảng dùng `delete[]`
Trả về tham chiếu tới private	Phá vỡ đóng gói – tránh
Gán mặc định	Sao chép từng thành viên – **sai** với lớp có con trỏ (sao chép nông)

Câu hỏi nhanh

- Lớp có `Time(int h, int m = 0)` – khai báo `Time t;` có hợp lệ không? → **Không** (h không có mặc định ⇒ không có constructor mặc định).
- Khi nào cần viết destructor? → Khi đối tượng giữ tài nguyên (bộ nhớ động, tập tin…).

4.2 Bài thực hành buổi 5 – Theo dõi thứ tự gọi constructor/destructor

Bài 1 – Thông báo trong constructor/destructor

- a) Chạy chương trình CreateAndDestroy (trang 2.3); **dự đoán trên giấy** trước, rồi đối chiếu kết quả.
- b) Sửa chương trình: gọi `Create()` **hai lần**; thêm đối tượng cấp phát động `new CreateAndDestroy(8)` trong main và **không** delete; thêm mảng 2 đối tượng. Giải thích kết quả mới.
- c) Thêm hàm `void F(CreateAndDestroy x)` (truyền giá trị) và gọi F(second): quan sát số lần destructor được gọi (giải thích ở buổi 8 – copy constructor).

Bài 2 – Lớp có cấp phát động

- a) Cài đặt lớp `Mang` (constructor nhận n, destructor giải phóng, Set/Get kiểm tra chỉ số, Nhap, Xuat, TongPhanTu).
- b) Viết lại lớp Time với constructor có tham số mặc định; thêm hàm TangPhut như ví dụ 5.6.
- c) Lớp `ChuoiKyTu` có thành viên `char *s`: constructor nhận `const char *`, cấp phát `strlen + 1` byte; destructor giải phóng; hàm `DoDai()`, `In()`.
- d) Biên dịch với `-fsanitize=address`, thử gán `b = a` với hai đối tượng Mang/ChuoiKyTu; ghi lại thông báo lỗi và giải thích.

4.3 Câu hỏi và bài tập

Câu hỏi

- 1) Nêu đặc điểm của constructor và destructor. Có thể đa năng hóa destructor không? Vì sao?
- 2) Cho biết thời điểm gọi constructor/destructor của đối tượng toàn cục, cục bộ, static, cấp phát động.
- 3) Vì sao hàm trả về tham chiếu tới thành viên private là nguy hiểm?
- 4) Phép gán mặc định giữa hai đối tượng làm gì? Khi nào gây lỗi?

Bài tập lập trình

- 1) Viết lớp `DongHo` đếm số đối tượng đang tồn tại bằng biến toàn cục: tăng trong constructor, giảm trong destructor; in số lượng sau mỗi bước của một chương trình có đối tượng cục bộ, mảng và new/delete.
- 2) Viết lớp `TepNhatKy` mở tập tin trong constructor (tên tập tin là tham số) và đóng trong destructor; hàm `Ghi(const char *)`. Kiểm chứng tập tin được đóng ngay cả khi hàm return sớm.

4.4 Bài tập dự đoán kết quả

```
class A
{  public:
      A(char c = 'x') { ch = c; cout << "C" << ch << " "; }
      ~A() { cout << "D" << ch << " "; }
   private:
      char ch;
};
A g('g');
void F() { static A s('s'); A t('t'); }
int main()
{  A a('a');
   F();  F();
   A *p = new A;
   { A b('b'); }
   delete p;
   cout << endl;
}
```

- **Đáp số:** `Cg Ca Cs Ct Dt Ct Dt Cx Cb Db Dx` rồi xuống dòng, sau đó `Da Ds Dg`.
- **Giải thích:**
  - **Cg** trước main; **Ca** đầu main.
  - Lần gọi F() thứ nhất: **Cs** (static, chỉ lần đầu), **Ct**, ra khỏi F: **Dt**. Lần thứ hai: chỉ **Ct Dt**.
  - `new A` ⇒ **Cx** (tham số mặc định 'x'); khối `{ A b('b'); }` ⇒ **Cb Db**; `delete p` ⇒ **Dx**.
  - main kết thúc: **Da**, rồi static **Ds**, cuối cùng **Dg**.